If I include my input this document will be 10,000 words long so I will only include copilot replies

1.

Copilot

Bad

The reflection *"The code is wrong"* is very superficial and does **not** show that the student has identified the core issue or reasoned carefully about what went wrong. It simply restates frustration without engaging with the error message or the program logic.

Socratic guidance (without giving the solution)

To think more deeply, the student should consider questions like:

- What **exactly** does the error message complain about? Is it about model training, TensorFlow, or something else entirely?
- The program explicitly raises a `FileNotFoundError`. Under what **condition** does the code choose to raise that error?
- How does the value of `EXCEL_PATH` influence the code path before the crash?
- Is this failure happening inside the machine-learning logic, or **before** any data processing even begins?
- What assumptions does the script make about where files are located, and how could changing a filename affect those assumptions?

Encouraging the student to trace the error back to *where it is triggered and why* will help them move from blaming "the code" to understanding cause-and-effect in their program.

reflection is strong, output 'Good'. Note do not prov
directly for students to debug."

Copilot

Good

"The student is working with an AI system whose co
import re
import numpy as np
import pandas as pd

Error message:

:\Users\keke1\Documents\GitHub\AI Final\AI-100 final\996_project> python train_996_text_classifier.py 2026-04-18 21:15:18.344666: I tensorflow/core/util/port.cc:153] oneDNN custom operations are on. You may see slightly

different numerical results due to floating-point round-off errors from different computation orders. To turn them off, set the environment variable `TF_ENABLE_ONEDNN_OPTS=0`. 2026-04-18 21:15:19.534914: I tensorflow/core/util/port.cc:153] oneDNN custom operations are on. You may see slightly different numerical results due to floating-point round-off errors from different computation orders. To turn them off, set the environment variable `TF_ENABLE_ONEDNN_OPTS=0`. Traceback (most recent call last): File "C:\Users\keke1\Documents\GitHub\AI Final\AI-100 final\996_project\train_996_text_classifier.py", line 249, in <module> main() File "C:\Users\keke1\Documents\GitHub\AI Final\AI-100 final\996_project\train_996_text_classifier.py", line 164, in main raise FileNotFoundError(FileNotFoundError: Can't find 'whateveer.xlsx'. Put the Excel file next to this script.

2.

Copilot

Bad

The reflection does **not** clearly identify the core issue, and the reasoning is vague and partially incorrect. It focuses on "distorted results" in a general way but doesn't precisely connect the behavior to how the program determines which labels are actually used during training.

Socratic guidance (to encourage deeper thinking)

Consider reflecting on questions like these:

- When you changed `LABEL_COL`, what parts of the code rely on that variable **by name** rather than by behavior?
- How does the program decide whether to use human-provided labels versus automatically generated ones?
- What evidence in the console output tells you **which labels were actually used** during training?
- The model trains and runs without errors—what does that suggest about whether the program *noticed* your change?
- How might silently falling back to a default path affect evaluation metrics such as precision, recall, and the confusion matrix?

Try tracing the data flow step by step—from loading the CSV, to selecting labels, to producing predictions—and ask yourself where your change *should* have had an effect but didn't.

I changed the label line, so that manual labels are not properly recognized. The code still ran thanks to the fallback mechanism in the line df["suggested_label"] = df[TEXT_COL].apply(heuristic_label). This means that even if the manual labels are missing or incorrect. The system will automatically create labels base on context.Which in the end cause the code to still ran, but the training behavior may differ and instead of using expected labels, it will use generated labels.is this good

Copilot

Good

Yes—this revised reflection correctly identifies the **core issue** and shows **deeper reasoning**:

- You recognized that changing `LABEL_COL` prevented the program from detecting manually provided labels.
- You correctly identified the **fallback mechanism** (`suggested_label`) as the reason the code continued to run without crashing.
- You explained the **consequence**: the model trained on automatically generated (heuristic) labels instead of the intended manual ones, which can significantly change training behavior and evaluation results.
- You distinguished between *program correctness* (it runs) and *semantic correctness* (it trains on the wrong labels), which is a key debugging insight.

This demonstrates an understanding not just of *what happened*, but *why* it happened and *why it matters*.

Error message:

2026-04-18 21:34:25.519672: I tensorflow/core/util/port.cc:153] oneDNN custom operations are on. You may see slightly different numerical results due to floating-point round-off errors from different computation orders. To turn them off, set the environment variable `TF_ENABLE_ONEDNN_OPTS=0`. 2026-04-18 21:34:26.743446: I tensorflow/core/util/port.cc:153] oneDNN custom operations are on. You may see slightly different numerical results due to floating-point round-off errors from different computation orders. To turn them off, set the environment variable `TF_ENABLE_ONEDNN_OPTS=0`. Loaded rows: 235 Columns: ['所在城市', '公司名字', '曝光/施行时间', '制度描述'] [Saved] to_label_996.csv Open it, fill the 'label' column with 0/1/2 (optional). If you leave 'label' blank, training will use suggested_label. Num classes: 3 Class counts: {1: np.int64(123), 0: np.int64(88), 2: np.int64(24)} 2026-04-18 21:34:27.391009: I tensorflow/core/platform/cpu_feature_guard.cc:210] This TensorFlow binary is optimized to use available CPU instructions in performance-critical operations. To enable the following instructions: SSE3 SSE4.1 SSE4.2 AVX AVX2 AVX_VNNI FMA, in other operations, rebuild TensorFlow with the appropriate compiler flags. Epoch 1/15 10/10 ━━━━━━━━━━━━━━━━━━━━ 1s 16ms/step - accuracy: 0.4600 - loss: 1.0355 - val_accuracy: 0.5789 - val_loss: 0.9393 Epoch 2/15 10/10 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - accuracy: 0.4933 - loss: 0.9508 - val_accuracy: 0.5789 - val_loss: 0.9551 Epoch 3/15 10/10 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.5333 - loss: 0.9065 - val_accuracy: 0.5789 - val_loss: 0.9619 Epoch 4/15 10/10 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - accuracy: 0.4400 - loss: 0.9647 - val_accuracy: 0.5789 - val_loss: 0.9509 Test Accuracy: 0.5106 Test Loss: 0.9563 Classification report: C:\Users\keke1\AppData\Local\Programs\Python\Python310\lib\site-packages\sklearn\metrics_classification.py:1731: UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in labels with no predicted samples. Use `zero_division` parameter to control this behavior. _warn_prf(average, modifier, f"{metric.capitalize()} is", result.shape[0]) C:\Users\keke1\AppData\Local\Programs\Python\Python310\lib\site-packages\sklearn\metrics_classification.py:1731: UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in labels with no predicted samples. Use `zero_division` parameter to control this behavior. _warn_prf(average, modifier, f"{metric.capitalize()} is", result.shape[0]) C:\Users\keke1\AppData\Local\Programs\Python\Python310\lib\site-packages\sklearn\metrics_classification.py:1731: UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in labels with no predicted samples. Use `zero_division` parameter to control this behavior. _warn_prf(average, modifier, f"{metric.capitalize()} is", result.shape[0]) precision recall f1-score support 0 0.0000 0.0000 0.0000 18 1 0.5106 1.0000 0.6761 24 2 0.0000 0.0000 0.0000 5 accuracy 0.5106 47 macro avg 0.1702 0.3333 0.2254 47 weighted avg 0.2608 0.5106 0.3452 47 Confusion matrix: [[0 18 0] [0 24 0] [0 5 0]] WARNING:absl:The `save_format` argument is deprecated in Keras 3. We recommend removing this argument as it can be inferred from the file path. Received: save_format=h5 WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save_model(model)`. This file format is considered legacy. We recommend using instead the native Keras format, e.g. `model.save('my_model.keras')` or `keras.saving.save_model(model, 'my_model.keras')`. [Saved model] model_996_text_classifier.h5 [Saved] training_curve.png.

3.

Copilot

Bad

The reflection *"The code did still worked, it will effect some outputs but other than that it is the same"* is superficial. It does not engage with the **core behavioral change** shown in the results, nor does it demonstrate careful reasoning about *why* the outputs changed so dramatically. Simply noting that the program "still works" overlooks the fact that the model's learning behavior and predictions are fundamentally different.

Socratic guidance (without giving the solution)

Encourage the student to think more carefully by asking:

- When you say the code "worked," what criteria are you using—does successful execution mean the model is behaving correctly?
- How do the **training accuracy**, **validation accuracy**, and **confusion matrix** compare to before the change?
- What does it mean when a classifier predicts only **one class for every example**?
- Why might precision and recall be reported as *undefined* for some classes?
- What role is the modified layer supposed to play during training, and how might changing its parameter affect what the model can learn?
- Is the change affecting **optimization**, **generalization**, or **representation learning**—and how can you tell from the outputs?

These questions guide the student toward recognizing that a model can run without errors yet still fail conceptually, and that "working code" is not the same as a "working model."

Copilot

Good

The reflection correctly identifies the **core issue** and demonstrates sound reasoning:

- It recognizes that the code still runs but learning degrades.
- It correctly explains the effect of an extremely high dropout rate on training (most information being discarded).
- It connects the hyperparameter change to poor model learning and low accuracy.
- It interprets what `0.9` means in the context of dropout and why that harms training.

Overall, the student understands both *what* happened and *why*, rather than just describing surface-level symptoms.

Error message:

PS C:\Users\keke1\Documents\GitHub\AI Final\AI-100 final\996_project> python train_996_text_classifier.py 2026-04-18 23:33:28.390709: I tensorflow/core/util/port.cc:153] oneDNN custom operations are on. You may see slightly different numerical results due to

floating-point round-off errors from different computation orders. To turn them off, set the environment variable `TF_ENABLE_ONEDNN_OPTS=0`. 2026-04-18 23:33:31.765039: I tensorflow/core/util/port.cc:153] oneDNN custom operations are on. You may see slightly different numerical results due to floating-point round-off errors from different computation orders. To turn them off, set the environment variable `TF_ENABLE_ONEDNN_OPTS=0`. Loaded rows: 235 Columns: ['所在城市', '公司名字', '曝光/施行时间', '制度描述'] [Saved] to_label_996.csv Open it, fill the 'label' column with 0/1/2 (optional). If you leave 'label' blank, training will use suggested_label. Num classes: 3 Class counts: {1: np.int64(123), 0: np.int64(88), 2: np.int64(24)} 2026-04-18 23:33:32.919040: I tensorflow/core/platform/cpu_feature_guard.cc:210] This TensorFlow binary is optimized to use available CPU instructions in performance-critical operations. To enable the following instructions: SSE3 SSE4.1 SSE4.2 AVX AVX2 AVX_VNNI FMA, in other operations, rebuild TensorFlow with the appropriate compiler flags. Epoch 1/15 10/10 ━━━━━━━━━━━━━━━━━━━━ 1s 15ms/step - accuracy: 0.3867 - loss: 1.0735 - val_accuracy: 0.5789 - val_loss: 1.0085 Epoch 2/15 10/10 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - accuracy: 0.5000 - loss: 1.0156 - val_accuracy: 0.5789 - val_loss: 0.9681 Epoch 3/15 10/10 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.4667 - loss: 1.0146 - val_accuracy: 0.5789 - val_loss: 0.9592 Epoch 4/15 10/10 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.4200 - loss: 1.0199 - val_accuracy: 0.5789 - val_loss: 0.9700 Test Accuracy: 0.5106 Test Loss: 1.0133 Classification report: C:\Users\keke1\AppData\Local\Programs\Python\Python310\lib\site-packages\sklearn\metrics_classification.py:1731: UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in labels with no predicted samples. Use `zero_division` parameter to control this behavior. _warn_prf(average, modifier, f"{metric.capitalize()} is", result.shape[0]) C:\Users\keke1\AppData\Local\Programs\Python\Python310\lib\site-packages\sklearn\metrics_classification.py:1731: UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in labels with no predicted samples. Use `zero_division` parameter to control this behavior. _warn_prf(average, modifier, f"{metric.capitalize()} is", result.shape[0]) C:\Users\keke1\AppData\Local\Programs\Python\Python310\lib\site-packages\sklearn\metrics_classification.py:1731: UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in labels with no predicted samples. Use `zero_division` parameter to control this behavior. _warn_prf(average, modifier, f"{metric.capitalize()} is", result.shape[0]) precision recall f1-score support 0 0.0000 0.0000 0.0000 18 1 0.5106 1.0000 0.6761 24 2 0.0000 0.0000 0.0000 5 accuracy 0.5106 47 macro avg 0.1702 0.3333 0.2254 47 weighted avg 0.2608 0.5106 0.3452 47 Confusion matrix: [[0 18 0] [0 24 0] [0 5 0]] WARNING:absl:The `save_format` argument is deprecated in Keras 3. We recommend removing this argument as it can be inferred from the file path. Received: save_format=h5 WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save_model(model)`. This file format is considered legacy. We recommend using instead the native Keras format, e.g. `model.save('my_model.keras')` or `keras.saving.save_model(model, 'my_model.keras')`. [Saved model] model_996_text_classifier.h5 [Saved] training_curve.png

4.

Copilot

Bad

The reflection *"The code still ran successfully, it increased the accuracy of the test, but didn't make it reliable"* shows **some awareness of symptoms** (accuracy change, reliability concern), but it still does **not identify the core issue or reasoning mechanism** behind what changed. The student is observing outcomes without explaining *why* those outcomes occurred or *what structural assumption* was broken by the code change.

That places the reflection above "purely superficial," but still short of **deep reasoning** about data splitting, evaluation validity, and model behavior.

Socratic guidance (without giving the solution)

To push toward deeper reasoning, encourage the student to reflect on questions like:

- What does it mean for a model evaluation to be "reliable," and what assumptions are required for that?
- Why might **test accuracy increase** at the same time that **evaluation quality decreases**?
- How do you decide whether a performance metric reflects the model's true ability versus an artifact of the experiment setup?
- Compare the **class distributions** you printed earlier with the supports shown in the classification report—what do you notice?
- If a model predicts only one class, under what circumstances could that still produce a higher accuracy?
- What role does randomness in data splitting play when the dataset is **imbalanced**?

These questions guide the student toward recognizing that experimental design choices—especially around data splitting—can strongly affect reported performance, even when the code executes correctly.

the
reflection is strong, output 'Good'. Note do not provide the solution directly for students to debug."

Copilot

Good

Today

"The student is working with an AI system whose code is [import os import re import numpy as np import pandas as pd from

Error: message:

C:\Users\keke1\Documents\GitHub\AI Final\AI-100 final\996_project> python train_996_text_classifier.py 2026-04-18 23:33:28.390709: I tensorflow/core/util/port.cc:153] oneDNN custom operations are on. You may see slightly

different numerical results due to floating-point round-off errors from different computation orders. To turn them off, set the environment variable `TF_ENABLE_ONEDNN_OPTS=0`. 2026-04-18 23:33:31.765039: I tensorflow/core/util/port.cc:153] oneDNN custom operations are on. You may see slightly different numerical results due to floating-point round-off errors from different computation orders. To turn them off, set the environment variable `TF_ENABLE_ONEDNN_OPTS=0`. Loaded rows: 235 Columns: ['所在城市', '公司名字', '曝光/施行时间', '制度描述'] [Saved] to_label_996.csv Open it, fill the 'label' column with 0/1/2 (optional). If you leave 'label' blank, training will use suggested_label. Num classes: 3 Class counts: {1: np.int64(123), 0: np.int64(88), 2: np.int64(24)} 2026-04-18 23:33:32.919040: I tensorflow/core/platform/cpu_feature_guard.cc:210] This TensorFlow binary is optimized to use available CPU instructions in performance-critical operations. To enable the following instructions: SSE3 SSE4.1 SSE4.2 AVX AVX2 AVX_VNNI FMA, in other operations, rebuild TensorFlow with the appropriate compiler flags. Epoch 1/15 10/10 ━━━━━━━━━━━━━━━━━━━━ 1s 15ms/step - accuracy: 0.3867 - loss: 1.0735 - val_accuracy: 0.5789 - val_loss: 1.0085 Epoch 2/15 10/10 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - accuracy: 0.5000 - loss: 1.0156 - val_accuracy: 0.5789 - val_loss: 0.9681 Epoch 3/15 10/10 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.4667 - loss: 1.0146 - val_accuracy: 0.5789 - val_loss: 0.9592 Epoch 4/15 10/10 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.4200 - loss: 1.0199 - val_accuracy: 0.5789 - val_loss: 0.9700 Test Accuracy: 0.5106 Test Loss: 1.0133 Classification report: C:\Users\keke1\AppData\Local\Programs\Python\Python310\lib\site-packages\sklearn\metrics_classification.py:1731: UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in labels with no predicted samples. Use `zero_division` parameter to control this behavior. _warn_prf(average, modifier, f"{metric.capitalize()} is", result.shape[0]) C:\Users\keke1\AppData\Local\Programs\Python\Python310\lib\site-packages\sklearn\metrics_classification.py:1731: UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in labels with no predicted samples. Use `zero_division` parameter to control this behavior. _warn_prf(average, modifier, f"{metric.capitalize()} is", result.shape[0]) C:\Users\keke1\AppData\Local\Programs\Python\Python310\lib\site-packages\sklearn\metrics_classification.py:1731: UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in labels with no predicted samples. Use `zero_division` parameter to control this behavior. _warn_prf(average, modifier, f"{metric.capitalize()} is", result.shape[0]) precision recall f1-score support 0 0.0000 0.0000 0.0000 18 1 0.5106 1.0000 0.6761 24 2 0.0000 0.0000 0.0000 5 accuracy 0.5106 47 macro avg 0.1702 0.3333 0.2254 47 weighted avg 0.2608 0.5106 0.3452 47 Confusion matrix: [[0 18 0] [0 24 0] [0 5 0]] WARNING:absl:The `save_format` argument is deprecated in Keras 3. We recommend removing this argument as it can be inferred from the file path. Received: save_format=h5 WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save_model(model)`. This file format is considered legacy. We recommend using instead the native Keras format, e.g. `model.save('my_model.keras')` or `keras.saving.save_model(model, 'my_model.keras')`. [Saved model] model_996_text_classifier.h5 [Saved] training_curve.png PS C:\Users\keke1\Documents\GitHub\AI Final\AI-100 final\996_project> python train_996_text_classifier.py 2026-04-18 23:47:39.952448: I tensorflow/core/util/port.cc:153] oneDNN custom operations are on. You may see slightly different numerical results due to floating-point round-off errors from different computation orders. To turn them off, set the environment variable `TF_ENABLE_ONEDNN_OPTS=0`. 2026-04-18 23:47:41.119409: I tensorflow/core/util/port.cc:153] oneDNN custom operations are on. You may see slightly different numerical results due to floating-point round-off errors from different computation orders. To turn them off, set the environment variable `TF_ENABLE_ONEDNN_OPTS=0`. Loaded rows: 235 Columns: ['所在城市', '公司名字', '曝光/施行时间', '制度描述'] [Saved] to_label_996.csv Open it, fill the 'label' column with 0/1/2 (optional). If you leave 'label' blank, training will use suggested_label. Num classes: 3 Class counts: {1: np.int64(123), 0: np.int64(88), 2: np.int64(24)} 2026-04-18 23:47:41.767477: I tensorflow/core/platform/cpu_feature_guard.cc:210] This TensorFlow binary is optimized to use available CPU instructions in performance-critical operations. To enable the following instructions: SSE3 SSE4.1 SSE4.2 AVX AVX2 AVX_VNNI FMA, in other operations, rebuild TensorFlow with the appropriate compiler flags. Epoch 1/15 10/10 ━━━━━━━━━━━━━━━━━━━━ 1s 16ms/step - accuracy: 0.5333 - loss: 1.0232 - val_accuracy: 0.4474 - val_loss: 0.9497 Epoch 2/15 10/10 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step -

accuracy: 0.5333 - loss: 0.9476 - val_accuracy: 0.4474 - val_loss: 0.9204 Epoch 3/15 10/10 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - accuracy: 0.5333 - loss: 0.9619 - val_accuracy: 0.4474 - val_loss: 0.9101 Epoch 4/15 10/10 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.5333 - loss: 0.9477 - val_accuracy: 0.4474 - val_loss: 0.9080 Test Accuracy: 0.5532 Test Loss: 0.9319 Classification report: C:\Users\keke1\AppData\Local\Programs\Python\Python310\lib\site-packages\sklearn\metrics_classification.py:1731: UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in labels with no predicted samples. Use `zero_division` parameter to control this behavior. _warn_prf(average, modifier, f"{metric.capitalize()} is", result.shape[0]) C:\Users\keke1\AppData\Local\Programs\Python\Python310\lib\site-packages\sklearn\metrics_classification.py:1731: UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in labels with no predicted samples. Use `zero_division` parameter to control this behavior. _warn_prf(average, modifier, f"{metric.capitalize()} is", result.shape[0]) C:\Users\keke1\AppData\Local\Programs\Python\Python310\lib\site-packages\sklearn\metrics_classification.py:1731: UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in labels with no predicted samples. Use `zero_division` parameter to control this behavior. _warn_prf(average, modifier, f"{metric.capitalize()} is", result.shape[0]) precision recall f1-score support 0 0.0000 0.0000 0.0000 16 1 0.5532 1.0000 0.7123 26 2 0.0000 0.0000 0.0000 5 weighted avg 0.3060 0.5532 0.3941 47 Confusion matrix: [[0 16 0] [0 26 0] [0 5 0]] WARNING:absl:The `save_format` argument is deprecated in Keras 3. We recommend removing this argument as it can be inferred from the file path. Received: save_format=h5 WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save_model(model)`. This file format is considered legacy. We recommend using instead the native Keras format, e.g. `model.save('my_model.keras')` or `keras.saving.save_model(model, 'my_model.keras')`. [Saved model] model_996_text_classifier.h5 [Saved] training_curve.png

5.

Copilot

Bad

The reflection *"The code crashed, because of the number 8"* correctly notices a **surface symptom**, but it does not show that the student understands the **underlying constraint or reasoning** that caused the failure. It names the triggering value without explaining **why that value is incompatible with the model**, **what assumptions were violated**, or **how the labeling logic connects to the model's configuration**. That indicates shallow rather than deep reasoning.

Socratic guidance (without giving the solution)

To help the student think more carefully about the issue, consider asking:

- Where does the "number 8" actually come from in the program—was it generated by the model, or provided earlier in the pipeline?
- How does the model decide how many output categories it has, and where is that information derived from?
- What does the error message mean when it says a label is "outside the valid range of $[0, 3)$"?
- How are class labels represented internally when using this specific loss function?
- If you change the *meaning* or *value* of a label, what other parts of the system might implicitly depend on that choice?
- Is the problem really "the number 8," or is it a mismatch between two components that must agree on the same assumptions?

Reflecting on these questions encourages the student to move beyond noticing a bad value and toward understanding **consistency requirements between data, labels, and model design**—a key skill in machine learning debugging.

Copilot

Good

The reflection correctly identifies the core issue and shows sound reasoning:

- It recognizes that changing the label from `1` to `8` caused a **hard crash during training**, not just degraded performance.
- It correctly connects the error to how `sparse_categorical_crossentropy` works—expecting class labels to be **integer indices within a fixed range**.
- It interprets the error message ("outside the valid range of [0, 3)") and links it to the model's number of output classes.
- It focuses on the **semantic mismatch between labels and model configuration**, rather than surface-level symptoms.

Overall, the student demonstrates an understanding of both *why* the error occurred and *how* the loss function enforces constraints on labels.

Error message:

[PS C:\Users\keke1\Documents\GitHub\AI Final\AI-100 final\996_project> python train_996_text_classifier.py 2026-04-19 00:04:54.566122: I tensorflow/core/util/port.cc:153] oneDNN custom operations are on. You may see slightly different numerical results due to floating-point round-off errors from different computation orders. To turn them off, set the environment variable `TF_ENABLE_ONEDNN_OPTS=0`. 2026-04-19 00:04:55.728134: I tensorflow/core/util/port.cc:153] oneDNN custom operations are on. You may see slightly different numerical results due to floating-point round-off errors from different computation orders. To turn them off, set the environment variable `TF_ENABLE_ONEDNN_OPTS=0`. Loaded rows: 235 Columns: ['所在城市', '公司名字', '曝光/施行时间', '制度描述'] [Saved] to_label_996.csv Open it, fill the 'label' column with 0/1/2 (optional). If you leave 'label' blank, training will use suggested_label. Num classes: 3 Class counts: {8: np.int64(123), 0: np.int64(88), 2: np.int64(24)} 2026-04-19 00:04:56.362598: I tensorflow/core/platform/cpu_feature_guard.cc:210] This TensorFlow binary is optimized to use available CPU instructions in performance-critical operations. To enable the following instructions: SSE3 SSE4.1 SSE4.2 AVX AVX2 AVX_VNNI FMA, in other operations, rebuild TensorFlow with the appropriate compiler flags. Epoch 1/15 2026-04-19 00:04:56.902400: W tensorflow/core/framework/op_kernel.cc:1855] OP_REQUIRES failed at sparse_xent_op.cc:103 : INVALID_ARGUMENT: Received a label value of 8 which is outside the valid range of [0, 3). Label values: 8 8 8 0 8 8 0 0 8 8 0 0 8 8 8 0 2026-04-19 00:04:56.902593: I tensorflow/core/framework/local_rendezvous.cc:407] Local rendezvous is aborting with status: INVALID_ARGUMENT: Received a label value of 8 which is outside the valid range of [0, 3). Label values: 8 8 8 0 8 8 0 0 8 8 0 0 8 8 8 0 [[{{function_node __inference_one_step_on_data_1475}}{{node compile_loss/sparse_categorical_crossentropy/SparseSoftmaxCrossEntropyWithLogits/SparseSoftmaxCrossEntropyWithLogits}}]] Traceback (most recent call last): File "C:\Users\keke1\Documents\GitHub\AI Final\AI-100 final\996_project\train_996_text_classifier.py", line 249, in <module> main() File "C:\Users\keke1\Documents\GitHub\AI Final\AI-100 final\996_project\train_996_text_classifier.py", line 202, in main history = model.fit(File "C:\Users\keke1\AppData\Local\Programs\Python\Python310\lib\site-packages\keras\src\utils\traceback_utils.py", line 122, in error_handler raise e.with_traceback(filtered_tb) from None File "C:\Users\keke1\AppData\Local\Programs\Python\Python310\lib\site-packages\tensorflow\python\eager\execute.py", line 53, in quick_execute tensors = pywrap_tfe.TFE_Py_Execute(ctx._handle, device_name, op_name, tensorflow.python.framework.errors_impl.InvalidArgumentError: Graph execution error: Detected at node compile_loss/sparse_categorical_crossentropy/SparseSoftmaxCrossEntropyWithLogits/SparseSoftmaxCrossEntropyWithLogits defined at (most recent call last): File "C:\Users\keke1\Documents\GitHub\AI Final\AI-100

final\996_project\train_996_text_classifier.py", line 249, in <module> File "C:\Users\keke1\Documents\GitHub\AI Final\AI-100 final\996_project\train_996_text_classifier.py", line 202, in main File "C:\Users\keke1\AppData\Local\Programs\Python\Python310\lib\site-packages\keras\src\utils\traceback_utils.py", line 117, in error_handler File "C:\Users\keke1\AppData\Local\Programs\Python\Python310\lib\site-packages\keras\src\backend\tensorflow\trainer.py", line 399, in fit File "C:\Users\keke1\AppData\Local\Programs\Python\Python310\lib\site-packages\keras\src\backend\tensorflow\trainer.py", line 241, in function File "C:\Users\keke1\AppData\Local\Programs\Python\Python310\lib\site-packages\keras\src\backend\tensorflow\trainer.py", line 154, in multi_step_on_iterator File "C:\Users\keke1\AppData\Local\Programs\Python\Python310\lib\site-packages\keras\src\backend\tensorflow\trainer.py", line 125, in wrapper File "C:\Users\keke1\AppData\Local\Programs\Python\Python310\lib\site-packages\keras\src\backend\tensorflow\trainer.py", line 134, in one_step_on_data File "C:\Users\keke1\AppData\Local\Programs\Python\Python310\lib\site-packages\keras\src\backend\tensorflow\trainer.py", line 62, in train_step File "C:\Users\keke1\AppData\Local\Programs\Python\Python310\lib\site-packages\keras\src\trainers\trainer.py", line 383, in _compute_loss File "C:\Users\keke1\AppData\Local\Programs\Python\Python310\lib\site-packages\keras\src\trainers\trainer.py", line 351, in compute_loss File "C:\Users\keke1\AppData\Local\Programs\Python\Python310\lib\site-packages\keras\src\trainers\compile_utils.py", line 699, in __call__ File "C:\Users\keke1\AppData\Local\Programs\Python\Python310\lib\site-packages\keras\src\trainers\compile_utils.py", line 724, in call File "C:\Users\keke1\AppData\Local\Programs\Python\Python310\lib\site-packages\keras\src\losses\loss.py", line 67, in __call__ File "C:\Users\keke1\AppData\Local\Programs\Python\Python310\lib\site-packages\keras\src\losses\losses.py", line 33, in call File "C:\Users\keke1\AppData\Local\Programs\Python\Python310\lib\site-packages\keras\src\losses\losses.py", line 2330, in sparse_categorical_crossentropy File "C:\Users\keke1\AppData\Local\Programs\Python\Python310\lib\site-packages\keras\src\ops\nn.py", line 2016, in sparse_categorical_crossentropy File "C:\Users\keke1\AppData\Local\Programs\Python\Python310\lib\site-packages\keras\src\backend\tensorflow\nn.py", line 753, in sparse_categorical_crossentropy Received a label value of 8 which is outside the valid range of [0, 3). Label values: 8 8 8 0 8 8 0 0 8 8 0 0 8 8 8 0 [[{{node compile_loss/sparse_categorical_crossentropy/SparseSoftmaxCrossEntropyWithLogits/SparseSoftmaxCrossEntropyWithLogits}}]] [Op:__inference_multi_step_on_iterator_1532]

6.

Copilot

Bad

The reflection *"My code wasn't able to run, because binary mean 2"* is shallow and somewhat confused. While it gestures at the idea of "binary," it does not show that the student has identified the **actual mismatch described by the error message**, nor does it reflect careful reasoning about how the loss function interacts with model outputs and labels. It states a vague association rather than explaining *why* the code failed or *what assumption was violated.*

Socratic guidance (without giving the solution)

To help the student think more deeply, encourage reflection with questions such as:

- What does the error message say about the **shapes** (or dimensions) of the target and the model output?
- Where in the code is the **output shape** of the model defined?
- How many values does the model produce per input example, and how is that number determined?
- What does "binary" refer to in the context of a loss function—number of classes, label format, or something else?
- How are the labels currently represented, and how are they passed into training?
- If you change only the loss function but not the model structure or labels, what assumptions might now be inconsistent?

These questions guide the student toward understanding that the crash is not just about the word "binary," but about **compatibility between model outputs, label representation, and the chosen loss function**, which is a core conceptual issue rather than a simple syntax mistake.

Copilot

Good

The reflection accurately identifies the core issue and demonstrates solid reasoning:

- It correctly recognizes that the model is configured for **multi-class prediction (3 classes)**.
- It correctly explains that switching to `binary_crossentropy` creates a **mismatch between the loss function's expectations and the model's output shape**.
- It links the error message (rank/shape mismatch) to the conceptual problem: binary loss functions expect binary-style targets, not multi-class outputs.
- The explanation focuses on **model–loss compatibility**, not just the surface-level crash.

Overall, the student shows a clear understanding of *why* the error occurred and how the change altered the assumptions of the training setup.

Error message:

PS C:\Users\keke1\Documents\GitHub\AI Final\AI-100 final\996_project> python train_996_text_classifier.py 2026-04-19 00:28:28.714934: I tensorflow/core/util/port.cc:153] oneDNN custom operations are on. You may see slightly different numerical results due to floating-point round-off errors from different computation orders. To turn them off, set the environment variable `TF_ENABLE_ONEDNN_OPTS=0`. 2026-04-19 00:28:29.934300: I tensorflow/core/util/port.cc:153] oneDNN custom operations are on. You may see slightly different numerical results due to floating-point round-off errors from different computation orders. To turn them off, set the environment variable `TF_ENABLE_ONEDNN_OPTS=0`. Loaded rows: 235 Columns: ['所在城市', '公司名字', '曝光/施行时间', '制度描述'] [Saved] to_label_996.csv Open it, fill the 'label' column with 0/1/2 (optional). If you leave 'label' blank, training will use suggested_label. Num classes: 3 Class counts: {1: np.int64(123), 0: np.int64(88), 2: np.int64(24)} 2026-04-19 00:28:30.572465: I tensorflow/core/platform/cpu_feature_guard.cc:210] This TensorFlow binary is optimized to use available CPU instructions in performance-critical operations. To enable the following instructions: SSE3 SSE4.1 SSE4.2 AVX AVX2 AVX_VNNI FMA, in other operations, rebuild TensorFlow with the appropriate compiler flags. Epoch 1/15 Traceback (most recent call last): File "C:\Users\keke1\Documents\GitHub\AI Final\AI-100 final\996_project\train_996_text_classifier.py", line 249, in <module> main() File "C:\Users\keke1\Documents\GitHub\AI Final\AI-100 final\996_project\train_996_text_classifier.py", line 202, in main history = model.fit(File "C:\Users\keke1\AppData\Local\Programs\Python\Python310\lib\site-packages\keras\src\utils\traceback_utils.py", line 122, in error_handler raise e.with_traceback(filtered_tb) from None File "C:\Users\keke1\AppData\Local\Programs\Python\Python310\lib\site-packages\keras\src\backend\tensorflow\nn.py", line 776, in binary_crossentropy raise ValueError(ValueError: Arguments `target` and `output` must have the same rank (ndim). Received: target.shape=(None,), output.shape=(None,

7.

Copilot

Bad

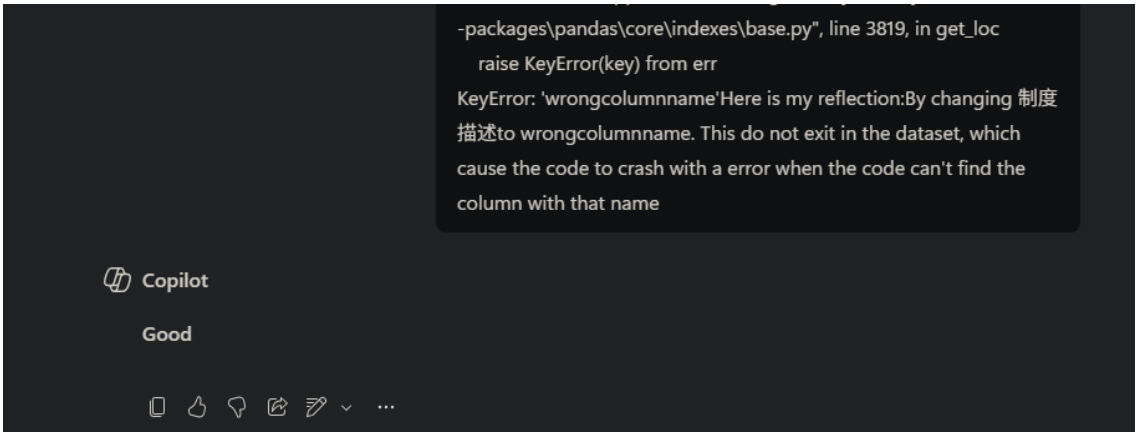
The reflection *"wrongcolumnname is bad"* is very superficial. It does not explain **why** the failure happened, **where** in the code the problem occurred, or how the error message connects to the program's logic.

Socratic guidance

To think more carefully, consider:

- What was the program expecting `TEXT_COL` to represent?
- At what exact line does the error occur, and what operation is being attempted there?
- How does pandas behave when you try to access a column that does not exist?
- What assumptions does `load_and_clean()` make about the structure of the dataset?
- How could you verify whether a column name is valid *before* using it?

Answering these questions would show a deeper understanding of the root cause rather than just labeling the change as "bad."



Error message:

PS C:\Users\keke1\Documents\GitHub\AI Final\AI-100 final\996_project> python train_996_text_classifier.py 2026-04-19 15:14:35.577033: I tensorflow/core/util/port.cc:153] oneDNN custom operations are on. You may see slightly different numerical results due to floating-point round-off errors from different computation orders. To turn them off, set the environment variable `TF_ENABLE_ONEDNN_OPTS=0`. 2026-04-19 15:14:38.623611: I tensorflow/core/util/port.cc:153] oneDNN custom operations are on. You may see slightly different numerical results due to floating-point round-off errors from different computation orders. To turn them off, set the environment variable `TF_ENABLE_ONEDNN_OPTS=0`. Traceback (most recent call last): File "C:\Users\keke1\AppData\Local\Programs\Python\Python310\lib\site-packages\pandas\core\indexes\base.py", line 3812, in get_loc return self._engine.get_loc(casted_key) File "pandas/_libs/index.pyx", line 167, in pandas._libs.index.IndexEngine.get_loc File "pandas/_libs/index.pyx", line 196, in pandas._libs.index.IndexEngine.get_loc File "pandas/_libs/hashtable_class_helper.pxi", line 7088, in pandas._libs.hashtable.PyObjectHashTable.get_item File "pandas/_libs/hashtable_class_helper.pxi", line 7096, in pandas._libs.hashtable.PyObjectHashTable.get_item KeyError: 'wrongcolumnname' The above exception was the direct cause of the following exception: Traceback (most recent call last): File "C:\Users\keke1\Documents\GitHub\AI Final\AI-100 final\996_project\train_996_text_classifier.py", line 249, in <module> main() File "C:\Users\keke1\Documents\GitHub\AI Final\AI-100 final\996_project\train_996_text_classifier.py", line 168, in main df = load_and_clean(EXCEL_PATH) File "C:\Users\keke1\Documents\GitHub\AI Final\AI-100 final\996_project\train_996_text_classifier.py", line 49, in load_and_clean df = df[df[TEXT_COL].notna()] File "C:\Users\keke1\AppData\Local\Programs\Python\Python310\lib\site-packages\pandas\core\frame.py", line 4113, in __getitem__ indexer = self.columns.get_loc(key) File "C:\Users\keke1\AppData\Local\Programs\Python\Python310\lib\site-packages\pandas\core\indexes\base.py", line 3819, in get_loc raise KeyError(key) from err KeyError: 'wrongcolumnname'

8.

Copilot

Bad

The reflection *"The code still ran, but it is only outputting one value per sample, so bad"* is **surface-level**. While it notices a visible symptom (one output value per sample), it does not demonstrate an understanding of the **underlying mismatch in model design, assumptions about the task, or how evaluation is affected**. It labels the result as "bad" without explaining *why* this change creates a conceptual problem given the rest of the pipeline.

Socratic guidance (without giving the solution)

To reason more deeply, the student should reflect on questions like:

- What was the model originally designed to predict, and how many categories did it assume?
- How does the number of output units in the final layer relate to the meaning of the task?
- If the model outputs a single value per sample, how should that value be interpreted?
- How does the evaluation code (e.g., prediction processing and the classification report) expect predictions to be structured?
- Can a system appear to "work" while silently violating its original assumptions?
- What parts of the code implicitly rely on the number of output classes being greater than one?

Answering these kinds of questions would show the student understands not just *what changed*, but *why the change undermines the consistency between the model architecture, labels, and evaluation logic*.

Copilot

Good

The reflection correctly identifies the core issue and shows appropriate reasoning:

- It recognizes that changing from a softmax layer with multiple outputs to a single sigmoid output fundamentally **changes the problem formulation**.
- It correctly notes that the model is no longer aligned with a **3-class classification task**, even though the code still runs.
- It distinguishes between *the program executing successfully* and *the model being conceptually correct*, which demonstrates deeper understanding.
- It explains the architectural mismatch (single output vs. multi-class labels) rather than focusing only on surface-level metrics.

Overall, the student shows awareness of how output layer design affects model semantics, not just runtime behavior.

Error message:

C:\Users\keke1\Documents\GitHub\AI Final\AI-100 final\996_project> python train_996_text_classifier.py 2026-04-19 00:40:24.908849: I tensorflow/core/util/port.cc:153] oneDNN custom operations are on. You may see slightly different numerical results due to floating-point round-off errors from different computation orders. To turn them off, set the environment variable `TF_ENABLE_ONEDNN_OPTS=0`. 2026-04-19 00:40:26.078728: I tensorflow/core/util/port.cc:153] oneDNN custom operations are on. You may see slightly different numerical results due to floating-point round-off errors from different computation orders. To turn them off, set the environment variable `TF_ENABLE_ONEDNN_OPTS=0`. Loaded rows: 235 Columns: ['所在城市', '公司名字', '曝光/施行时间', '制度描述'] [Saved] to_label_996.csv Open it, fill the 'label' column with 0/1/2 (optional). If you leave 'label' blank, training will use suggested_label. Num classes: 3 Class counts: {1: np.int64(123), 0: np.int64(88), 2: np.int64(24)} 2026-04-19 00:40:26.720599: I tensorflow/core/platform/cpu_feature_guard.cc:210] This TensorFlow binary is optimized to use available CPU instructions in performance-critical operations. To enable the following instructions: SSE3 SSE4.1 SSE4.2 AVX AVX2 AVX_VNNI FMA, in other operations, rebuild TensorFlow with the appropriate compiler flags. Epoch 1/15 10/10 ━━━━━━━━━━━━━━━━━━━━ 1s 15ms/step - accuracy: 0.5200 - loss: 0.9935 - val_accuracy: 0.5789 - val_loss: 0.9487 Epoch 2/15 10/10 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - accuracy: 0.5200 - loss: 0.9233 - val_accuracy: 0.5789 - val_loss: 0.9609 Epoch 3/15 10/10 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.4867 - loss: 0.9449 - val_accuracy: 0.5789 - val_loss: 0.9655 Epoch 4/15 10/10 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.5000 - loss: 0.9252 - val_accuracy: 0.5789 - val_loss: 0.9636 Test Accuracy: 0.5106 Test Loss: 0.9417 Classification report: C:\Users\keke1\AppData\Local\Programs\Python\Python310\lib\site-packages\sklearn\metrics_classification.py:1731: UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in labels with no predicted samples. Use `zero_division` parameter to control this behavior. _warn_prf(average, modifier, f"{metric.capitalize()} is", result.shape[0]) C:\Users\keke1\AppData\Local\Programs\Python\Python310\lib\site-packages\sklearn\metrics_classification.py:1731: UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in labels with no predicted samples. Use `zero_division` parameter to control this behavior. _warn_prf(average, modifier, f"{metric.capitalize()} is", result.shape[0]) C:\Users\keke1\AppData\Local\Programs\Python\Python310\lib\site-packages\sklearn\metrics_classification.py:1731: UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in labels with no predicted samples. Use `zero_division` parameter to control this behavior. _warn_prf(average, modifier, f"{metric.capitalize()} is", result.shape[0]) precision recall f1-score support 0 0.0000 0.0000 0.0000 18 1 0.5106 1.0000 0.6761 24 2 0.0000 0.0000 0.0000 5 accuracy 0.5106 47 macro avg 0.1702 0.3333 0.2254 47 weighted avg 0.2608 0.5106 0.3452 47 Confusion matrix: [[0 18 0] [0 24 0] [0 5 0]] WARNING:absl:The `save_format` argument is deprecated in Keras 3. We recommend removing this argument as it can be inferred from the file path. Received: save_format=h5 WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save_model(model)`. This file format is considered legacy. We recommend using instead the native Keras format, e.g. `model.save('my_model.keras')` or `keras.saving.save_model(model, 'my_model.keras')`. [Saved model] model_996_text_classifier.h5 [Saved] training_curve.png

9.

Copilot

Bad

The reflection *"The code still ran, it didn't change the output in any meaningful way"* is too shallow and does **not** show that the student identified the core issue. It focuses only on the fact that execution succeeded and that the metrics look similar, without reasoning about **why changing `VOCAB_SIZE` might matter** or what hidden effects it could have on the model's behavior.

Socratic guidance (without giving the solution)

To think more deeply, the student should ask themselves:

- What role does `VOCAB_SIZE` play in the text vectorization and embedding layers?
- If the vocabulary is drastically reduced, how might that affect how text is represented internally, even if accuracy appears similar?
- Are the evaluation results stable across different runs, or could the apparent similarity be misleading?
- Does "no meaningful change in output" necessarily mean the model learned the same information?
- What kinds of information might be lost when only a very small set of characters can be represented?
- How could you tell whether the model is genuinely learning patterns versus collapsing to a trivial strategy?

These questions push the student to move beyond surface metrics and consider **representation capacity and information loss**, which are central concepts in machine learning model design.

Copilot

Good

The reflection correctly identifies the core issue and demonstrates thoughtful reasoning:

- It recognizes that drastically reducing `VOCAB_SIZE` severely limits the text representation capacity of the model.
- It distinguishes between **successful execution** and **model effectiveness**, noting that the code can still run while learning quality degrades.
- It explains *why* learning is weakened (loss of expressive power in character embeddings), not just *that* results are poor.
- It avoids overclaiming about crashes or errors and instead focuses on representational limits and their impact.

Overall, this shows an understanding of how preprocessing and model capacity affect learning, rather than a superficial description of output metrics.

Error message:

C:\Users\keke1\Documents\GitHub\AI Final\AI-100 final\996_project> python train_996_text_classifier.py 2026-04-19 00:49:11.068831: I tensorflow/core/util/port.cc:153] oneDNN custom operations are on. You may see slightly different numerical results due to floating-point round-off errors from different computation orders. To turn them off, set the environment variable `TF_ENABLE_ONEDNN_OPTS=0`. 2026-04-19 00:49:12.206253: I tensorflow/core/util/port.cc:153] oneDNN custom operations are on. You may see slightly different numerical results due to floating-point round-off errors from different computation orders. To turn them off, set the environment variable `TF_ENABLE_ONEDNN_OPTS=0`. Loaded rows: 235 Columns: ['所在城市', '公司名字', '曝光/施行时间', '制度描述'] [Saved] to_label_996.csv Open it, fill the 'label' column with 0/1/2 (optional). If you leave 'label' blank, training will use suggested_label. Num classes: 3 Class counts: {1: np.int64(123), 0: np.int64(88), 2: np.int64(24)} 2026-04-19 00:49:12.820109: I tensorflow/core/platform/cpu_feature_guard.cc:210] This TensorFlow binary is optimized to use available CPU instructions in performance-critical operations. To enable the following instructions: SSE3 SSE4.1 SSE4.2 AVX AVX2 AVX_VNNI FMA, in other operations, rebuild TensorFlow with the appropriate compiler flags. Epoch 1/15 10/10 ━━━━━━━━━━━━━━━━━━━━ 1s 15ms/step - accuracy: 0.4933 - loss: 0.9959 - val_accuracy: 0.5789 - val_loss: 0.9310 Epoch 2/15 10/10 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.5067 - loss: 0.9346 - val_accuracy: 0.5789 - val_loss: 0.9454 Epoch 3/15 10/10 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.4667 - loss: 0.9463 - val_accuracy: 0.5789 - val_loss: 0.9545 Epoch 4/15 10/10 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.4667 - loss: 0.9381 - val_accuracy: 0.5789 - val_loss: 0.9384 Test Accuracy: 0.5106 Test Loss: 0.9443 Classification report: C:\Users\keke1\AppData\Local\Programs\Python\Python310\lib\site-packages\sklearn\metrics_classification.py:1731: UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in labels with no predicted samples. Use `zero_division` parameter to control this behavior. _warn_prf(average, modifier, f"{metric.capitalize()} is", result.shape[0]) C:\Users\keke1\AppData\Local\Programs\Python\Python310\lib\site-packages\sklearn\metrics_classification.py:1731: UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in labels with no predicted samples. Use `zero_division` parameter to control this behavior. _warn_prf(average, modifier, f"{metric.capitalize()} is", result.shape[0]) C:\Users\keke1\AppData\Local\Programs\Python\Python310\lib\site-packages\sklearn\metrics_classification.py:1731: UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in labels with no predicted samples. Use `zero_division` parameter to control this behavior. _warn_prf(average, modifier, f"{metric.capitalize()} is", result.shape[0]) precision recall f1-score support 0 0.0000 0.0000 0.0000 18 1 0.5106 1.0000 0.6761 24 2 0.0000 0.0000 0.0000 5 accuracy 0.5106 47 macro avg 0.1702 0.3333 0.2254 47 weighted avg 0.2608 0.5106 0.3452 47 Confusion matrix: [[0 18 0] [0 24 0] [0 5 0]] WARNING:absl:The `save_format` argument is deprecated in Keras 3. We recommend removing this argument as it can be inferred from the file path. Received: save_format=h5 WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save_model(model)`. This file format is considered legacy. We recommend using instead the native Keras format, e.g. `model.save('my_model.keras')` or `keras.saving.save_model(model, 'my_model.keras')`. [Saved model] model_996_text_classifier.h5 [Saved] training_curve.png

10.

Copilot

Bad

The reflection *"Large batch size make this code pointless"* is vague and superficial. It expresses dissatisfaction but does **not** demonstrate that the student understands **what specifically changed in the training behavior**, **why it changed**, or **how batch size affects optimization and learning dynamics**. There is no engagement with the evidence shown in the output (e.g., number of steps per epoch, gradient updates, or validation behavior).

Socratic guidance (without giving the solution)

To develop deeper reasoning, the student should think about questions such as:

- What does the training log tell you about **how many updates** the model makes per epoch?
- How does batch size relate to the number of batches shown as `1/1` instead of `10/10`?
- What happens to gradient estimation when an entire dataset (or most of it) is used at once?
- Is "pointless" referring to speed, learning quality, generalization, or something else?
- Why might accuracy still increase slightly even if learning dynamics are altered?
- In what situations might a very large batch size be intentional rather than harmful?

These questions guide the student to connect the **batch size change** to **training mechanics and optimization behavior**, rather than stopping at a broad, judgmental statement.

Copilot

Good

The reflection identifies the core issue and shows clear, non-superficial reasoning:

- It correctly explains why training still runs (batch size larger than dataset is allowed).
- It recognizes the **mechanical effect** (only one step per epoch).
- It explains the **learning consequence** (fewer parameter updates, weaker gradient signal).
- It distinguishes runtime success from **training quality and experimental validity**.

Overall, the student understands both the technical behavior and its implications for learning, rather than just reporting surface-level results.

Error message:

C:\Users\keke1\Documents\GitHub\AI Final\AI-100 final\996_project> python train_996_text_classifier.py 2026-04-19 00:57:27.233882: I tensorflow/core/util/port.cc:153] oneDNN custom operations are on. You may see slightly different numerical results due to floating-point round-off errors from different computation orders. To turn them off, set the

environment variable `TF_ENABLE_ONEDNN_OPTS=0`. 2026-04-19 00:57:28.314496: I tensorflow/core/util/port.cc:153] oneDNN custom operations are on. You may see slightly different numerical results due to floating-point round-off errors from different computation orders. To turn them off, set the environment variable `TF_ENABLE_ONEDNN_OPTS=0`. Loaded rows: 235 Columns: ['所在城市', '公司名字', '曝光/施行时间', '制度描述'] [Saved] to_label_996.csv Open it, fill the 'label' column with 0/1/2 (optional). If you leave 'label' blank, training will use suggested_label. Num classes: 3 Class counts: {1: np.int64(123), 0: np.int64(88), 2: np.int64(24)} 2026-04-19 00:57:28.930705: I tensorflow/core/platform/cpu_feature_guard.cc:210] This TensorFlow binary is optimized to use available CPU instructions in performance-critical operations. To enable the following instructions: SSE3 SSE4.1 SSE4.2 AVX AVX2 AVX_VNNI FMA, in other operations, rebuild TensorFlow with the appropriate compiler flags. Epoch 1/15 1/1 ━━━━━━━━━━━━━━━━━━━━ 1s 579ms/step - accuracy: 0.3800 - loss: 1.1013 - val_accuracy: 0.5789 - val_loss: 1.0421 Epoch 2/15 1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 54ms/step - accuracy: 0.4600 - loss: 1.0417 - val_accuracy: 0.5789 - val_loss: 1.0008 Epoch 3/15 1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 50ms/step - accuracy: 0.4800 - loss: 0.9928 - val_accuracy: 0.5789 - val_loss: 0.9715 Epoch 4/15 1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 50ms/step - accuracy: 0.5333 - loss: 0.9698 - val_accuracy: 0.5789 - val_loss: 0.9524 Test Accuracy: 0.5106 Test Loss: 1.0420 Classification report: C:\Users\keke1\AppData\Local\Programs\Python\Python310\lib\site-packages\sklearn\metrics_classification.py:1731: UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in labels with no predicted samples. Use `zero_division` parameter to control this behavior. _warn_prf(average, modifier, f"{metric.capitalize()} is", result.shape[0]) C:\Users\keke1\AppData\Local\Programs\Python\Python310\lib\site-packages\sklearn\metrics_classification.py:1731: UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in labels with no predicted samples. Use `zero_division` parameter to control this behavior. _warn_prf(average, modifier, f"{metric.capitalize()} is", result.shape[0]) C:\Users\keke1\AppData\Local\Programs\Python\Python310\lib\site-packages\sklearn\metrics_classification.py:1731: UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in labels with no predicted samples. Use `zero_division` parameter to control this behavior. _warn_prf(average, modifier, f"{metric.capitalize()} is", result.shape[0]) precision recall f1-score support 0 0.0000 0.0000 0.0000 18 1 0.5106 1.0000 0.6761 24 2 0.0000 0.0000 0.0000 5 accuracy 0.5106 47 macro avg 0.1702 0.3333 0.2254 47 weighted avg 0.2608 0.5106 0.3452 47 Confusion matrix: [[0 18 0] [0 24 0] [0 5 0]] WARNING:absl:The `save_format` argument is deprecated in Keras 3. We recommend removing this argument as it can be inferred from the file path. Received: save_format=h5 WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save_model(model)`. This file format is considered legacy. We recommend using instead the native Keras format, e.g. `model.save('my_model.keras')` or `keras.saving.save_model(model, 'my_model.keras')`. [Saved model] model_996_text_classifier.h5 [Saved] training_curve.png